Mathematical modelling of T-Adaptive Segment Routing

Project Orange — Challenge EURO/ROADEF 2026

Nguyễn Tiến Đức (20252076M) · Hoàng Huy Chiến (20261069M)

IT4033E Capstone, SoICT · October 2026

Problem: T-Adaptive Segment Routing

- IP backbone G = (V, A): link metric ω(a), capacity c(a); demands d = (s_d, t_d) with volume ν(d, t)
- Packets follow shortest paths; ties are split (ECMP). Congestion = high load λ(a, t) = flow(a, t) / c(a)
- **Segment Routing:** each demand gets at most maxSeg − 1 waypoints; shortest paths between consecutive waypoints bend flows away from hot links
- **Periods:** t = 0 nominal, t = 1 with links q(1) down; at most κ(1) segment changes between periods

Goal choose a segment path per demand and period to minimise link load. Experiments: period 0 alone, then periods {0, 1} with the budget, on the 20 instances of set A.

Shortest path crosses a hot link
Waypoint w bends the flow away

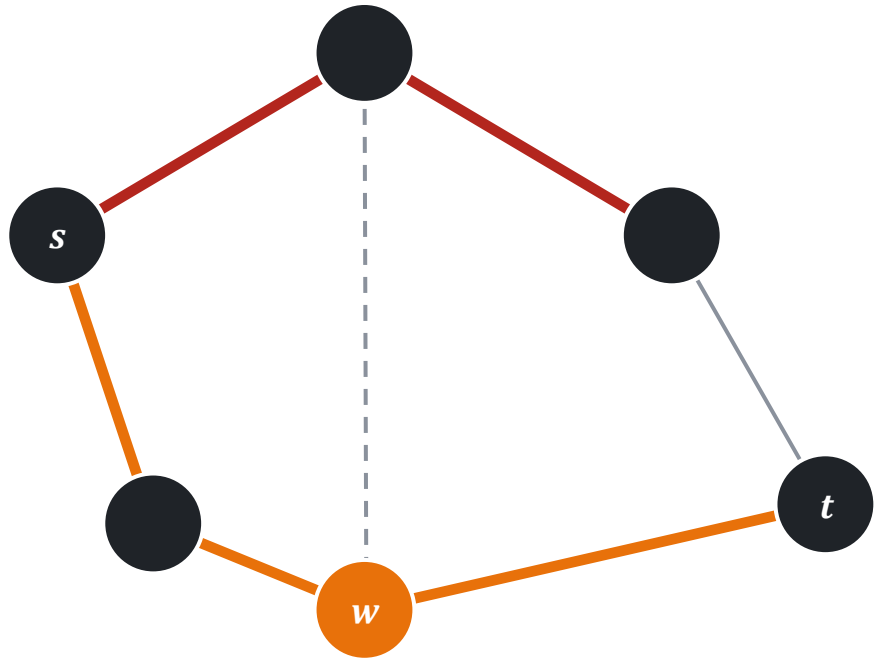


Illustration only, not an instance from the data

Data and split coefficients r(i, j, a, t)

- **Input:** net.json (ω, c), tm.json (v), scenario.json (maxSeg, κ, q)
- r(i, j, a, t) = fraction of a unit flow i → j in G(t) that crosses link a (course document)
- σ = number of shortest paths: one Dijkstra with path counting per source, O(n·m·log n)

$$r(i, j, a, t) = \sigma_{iu}(t) \cdot \sigma_{vj}(t) \,/\, \sigma_{ij}(t) \quad \text{if } d_{iu} + \omega(a) + d_{vj} = d_{ij},\ a = (u, v);\ \text{else } 0$$

Path counting ≠ per-node ECMP: toy instance, r(v0, v5, a13, 0)

1/3

path counting (course definition, kept)

1/2

per-node ECMP (subject's table)

We keep the course definition and document the difference in the report.

MILP model

Variables and objective

- $x^{dt}_{ij} \in \{0, 1\}$: segment $(i, j) \in K$ used by demand d in period t (K = reachable pairs)
- $U \geq 0$: maximum link utilisation (MLU)
- $y^{dt}_{ij} \geq 0$ linearises $|x^{dt}_{ij} - x^{d,t-1}_{ij}|$
- **Objective: min U** (first criterion of the subject's lexicographic objective)

(1) Path in the segment graph

$$\sum_j x^{dt}_{ij} - \sum_j x^{dt}_{ji} = 1[i = s_d] - 1[i = t_d]$$

(2) Segment limit

$$\sum_{(i,j)} x^{dt}_{ij} \leq \text{maxSeg}$$

(3) Load on every working link

$$\sum_d \sum_{(i,j)} r(i, j, a, t) \cdot \nu(d, t) \cdot x^{dt}_{ij} \leq U \cdot c(a), \quad a \notin q(t)$$

(4) Change budget

$$\sum y^{dt}_{ij} \leq \kappa(t), \quad y \geq x^t - x^{t-1}, \quad y \geq x^{t-1} - x^t$$

Size $|D| \cdot |K| \cdot |T|$ binaries with $|K| \leq n(n-1)$: from $1.5 \cdot 10^4$ (instance 01) to $\approx 10^9$ (instance 20)

Implementation and validation

Stack
Julia 1.13 + JuMP 1.32 + HiGHS 1.15 (open-source branch-and-bound)

Validation first
24 automated tests on the subject's toy instance; MILP optimum = brute-force enumeration of all segment paths

MIP start
No-waypoint routing (segment $s_d \rightarrow t_d$ only) is always feasible with 0 changes; its MLU is the reference column no-wp

Protocol
10 runs per instance (HiGHS seeds 1–10, 1 thread each); limit 1800 s (period 0) / 3600 s (periods 0+1); gap = 1 − LB/UB; CPU = optimize! time only

Machine
i9-14900K, 125 GB RAM (shared server); parallel runs through a memory-aware job queue

24

automated tests pass

0.375

toy optimum MLU, waypoint v4: MILP = brute force

10

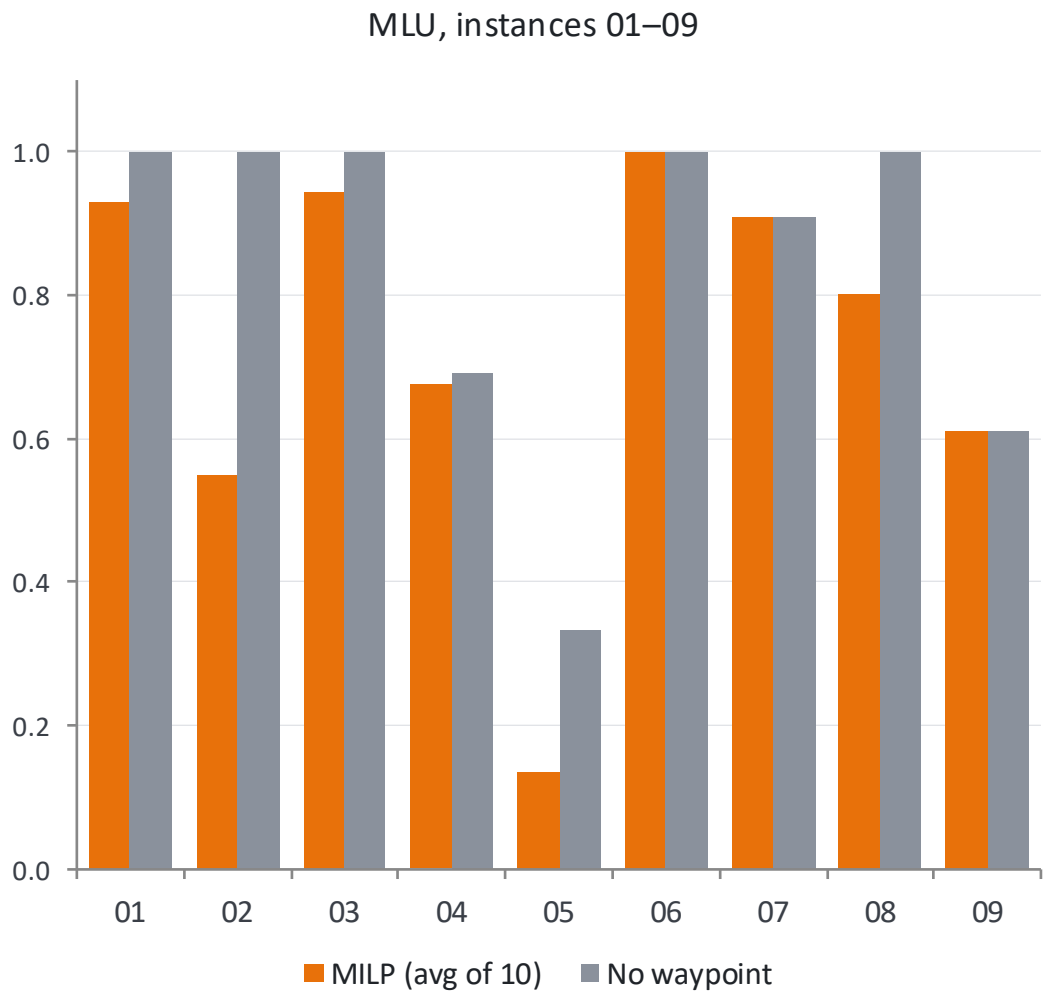
seeded runs per instance

Results — period 0 (averages over 10 runs)

Inst	n	m	|D|	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.02M	**10/10**	**0.929**	1.000	0.0	4.0	1.0
02	30	150	45	0.04M	**10/10**	**0.549**	1.000	0.0	27.8	1.0
03	50	250	20	0.05M	**10/10**	**0.944**	1.000	0.0	16.3	1.0
04	50	250	200	0.49M	0/10	**0.675**	0.692	31.5	1819.3	0.0
05	100	396	100	0.99M	4/10	**0.135**	0.333	12.6	1578.5	0.4
06	100	500	500	4.95M	0/10	1.000	1.000	100.0	2056.4	0.0
07	100	500	800	7.92M	2/10	0.908	0.908	80.0	2069.4	0.2
08	150	654	200	4.47M	1/10	**0.800**	1.000	53.8	12450.4	0.1
09	150	750	200	4.47M	0/10	0.611	0.611	17.3	2403.3	0.0
10	150	966	1000	22.35M	0/10	1.000	1.000	100.0	2989.6	0.0
11	200	1000	400	15.92M	0/10	1.000	1.000	100.0	2430.3	0.0
12	200	898	400	15.92M	0/10	1.000	1.000	100.0	2342.9	0.0
14	250	1108	600	37.35M	0/10	1.000	1.000	100.0	3032.5	0.0
15	250	1250	600	37.35M	0/10	1.000	1.000	100.0	2826.6	0.0
13, 16–20: model too large to build in 125 GB (79.6M – 957.6M variables)										

Green MLU = waypoints beat the no-waypoint routing. Time limit 1800 s.

Period 0 — observations



- **01–03:** optimal at the root node in seconds; MLU 1.00 → 0.93, 0.55, 0.94
- **04–05:** no proof in 30 min, all time in the root node. Strong seed effect: on 05, 4/10 seeds prove 0.1155; on 04 only 1/10 improves the MIP start → average over 10 seeds
- **06–12:** millions of binaries, LP relaxation takes most of the time. 08: 1 seed proves 0.319, 3 stop at 0.561, 6 keep 1.000. 10–12, 14, 15: MIP start, no lower bound (gap 100%)
- **Time limit:** HiGHS can overrun it, up to 51 940 s for an 1800 s limit (08)
- **13–20:** up to $9.6 \cdot 10^8$ binaries; ≈ 2 GB per million variables → only 14, 15 fit in 125 GB

Results — periods 0 and 1 (budget, presolve off)

Inst	n	m	|D|	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.05M	**10/10**	**0.929**	1.000	0.0	23.4	11.0
02	30	150	45	0.12M	7/10	**0.942**	1.098	5.2	2386.5	6.1
03	50	250	20	0.15M	3/10	**0.961**	1.000	24.0	3109.1	44.4
04	50	250	200	1.47M	0/10	**0.898**	1.000	32.9	3638.6	0.0
05	100	396	100	2.97M	0/10	**0.865**	1.000	77.1	5198.7	0.0
06	100	500	500	14.85M	0/5	1.000	1.000	100.0	3893.4	0.0
08	150	654	200	13.41M	0/1	1.000	1.000	68.1	3892.2	0.0
09	150	750	200	13.41M	0/5	1.000	1.000	66.0	3869.9	0.0

07: ≈ 88 GB, can only run alone (not in this record). 10–20: model too large to build (47.8M – 2872.8M variables)

Time limit 3600 s. Rows with fewer than 10 runs (06, 08, 09) were cut by time and memory limits; the report states this.

1.098 → 0.942

instance 02: link down congests the network; waypoints bring average MLU below 1

7/10

seeds prove the optimum 0.903 on 02 (2/10 with presolve on)

× 3

variables vs. period 0: two copies of x plus y

Two periods — observations

Presolve stall

1 **Symptom**
HiGHS defaults: instance 04 stayed in presolve > 3 h for a 1 h limit

2 **Ablation (60 s limit)**
remove constraint (4) or turn presolve off → the limit is respected

3 **Decision**
the dense budget row blocks presolve → all two-period runs use presolve off

- **Instance 02:** the failed link makes the no-waypoint MLU 1.098 (congestion); optimum 0.903 proved by 7/10 seeds with presolve off vs. 2/10 with presolve on
- **Instance 05:** 2/10 seeds find 0.323 (no-wp 1.000); the others keep the MIP start
- **Price of presolve off:** memory grows to 3.7–5.5 GB per million variables → model of 07 (≈ 88 GB) only alone; 10–20 cannot be built

Experimental process: trial and error

Symptom	Diagnosis	Fix / consequence
Toy: r = 1/3, subject says 1/2	path counting ≠ per-node ECMP	**keep the course definition**
04: no feasible solution in 300 s	stuck in the root node	**no-waypoint MIP start**
Laptop 24 GB too small	measured ≈ 2–3 GB per million variables	**125 GB server + memory-aware queue**
Two-period build took 5428 s	sum over a JuMP array is quadratic	**generator form: 1.4 s**
optimize! > 3 h with a 1 h limit	ablation: presolve × budget row	**presolve off for two periods**
Runs far beyond the time limit	HiGHS checks the limit between steps (likely)	**report measured time; reorder queue**

Lessons

Validate on a toy instance against brute force first. · Solver settings are part of the protocol. · For exact MILPs of this size, memory is the first limit, before time.

Conclusion

$1.00 \rightarrow 0.55$

MLU on instance 02 (period 0), proved optimal

$\approx 10^6$

binaries: beyond this the solver rarely improves the MIP start

$\approx 6 \cdot 10^7$

variables (period 0): beyond this the model cannot be built in 125 GB

- Compact MILP for T-ASR (segment variables, path-counting split ratios), validated against brute force
- Exact solving works for small instances and shows large MLU reductions
- **Next steps:** restrict candidate waypoints, column generation on segment paths, full lexicographic objective

Thank you — questions?

Backup — periods 0 and 1 with presolve on

Inst	n	m	|D|	vars	opt/runs	MLU	no-wp	gap (%)	CPU (s)	nodes
01	20	80	40	0.05M	**10/10**	**0.929**	1.000	0.0	32.6	1.0
02	30	150	45	0.12M	2/10	**1.053**	1.098	42.9	3224.8	0.7
03	50	250	20	0.15M	4/10	**0.946**	1.000	11.5	2840.8	0.8
Only instances 01–03 were run with presolve on; on 04 presolve stalled for more than 3 h										

2/10

optimal on 02 with presolve on

7/10

optimal on 02 with presolve off